



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Санах ойн менежмент

Лекц 5

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- C/O-н менежментийн аргууд
- C/O-н статик менежмент
- Стек дээрх c/o-н динамик менежмент
 - Идэвхжлийн бичлэг
 - Стэк менежмент
- Овоолго дээрх динамик менежмент
 - Тогтмол хэмжээтэй блок
 - Хувьсах хэмжээтэй блок



Санах ойн менежмент хийх нь хийсвэр машины интерпретаторын чухал бүрдэл хэсэг болдог.

Арга техникүүд: Статик/динамик менежмент, Идэвхжлийн бичлэг (Activation record), Стэк (Stack)/Овоолго (Heap) хэрэглэх, (Хамрах хүрээний дүрмийн хэрэгжилтүүлэлтийн дата бүтэц, механизмууд – дараагийн лекц)

Garbage-collection: Овоолго дээрх СО хуваарилалтын автомат сэргээлт

**Физик машины хувьд үнэхээр энгийн байвал дээд түвшний
хэлний хийсвэр машины хувьд харьцангуй бэрхшээлтэй**

Программ, өгөгдөлд зориулсан санах ойн хуваарилалтыг удирддаг:

Санах ойд ямар эрэмбээр байрлах; Хэр их хугацаанд байрших; СО-оос мэдээлэл авахад ямар туслах бүтэц шаардлагатайг тодорхойлдог.

- **Доод түвшний хийсвэр машин:** Маш энгийн, бүхэлдээ *статик*
 - Програмын биелэлт эхлэхээс өмнө
 - Машин хэл дээрх программ, датаг санах ойд ачаалан дуусах хүртэл хадгална
- **Дээд түвшний хийсвэр машин:** Статик хангалтгүй, *динамик*
 - Рекурс: Идэвхтэй зэрэгцээ дуудалт – параметрууд, завсрын үр дүн, буцах хаяг
- **Стэк – LIFO (Last In First Out):** Биелэлтийн явцад хуваарилах, чөлөөлөх
- **Овоолго (Heap) – Ил (explicit) хуваарилалт/чөлөөлөлт**
 - Жнь: С хэлний malloc, free нь дараалал харгалзахгүй

СО-н статик менежмент

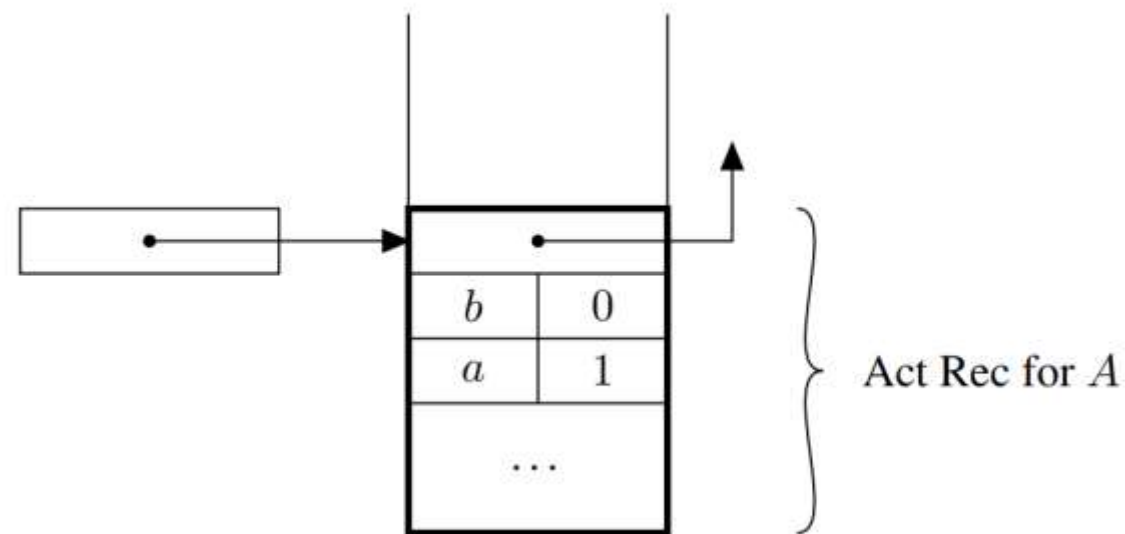
- Биелэлтээс өмнө санах ойн тогтмол бүсэд компилятор хадгална
 - *Глобал хувьсагч*
 - *Объект кодын заавар*: программ биелэгдэх явцад өөрчлөгддөггүй
 - *Тогтмол*: компиляцын явцад өөр үл мэдэгдэгчээс хамаараагүй бол
 - *Compiler-Generated tables*: хэлийг ажиллуулахад шаардагдах мэдээллүүд (нэрийг хянан барьж байх, төрөл шалгах, хаягдал цуглуулах ...)
- Хэл нь рекурс дэмждэггүй бол хэлний бусад бүрдэл хэсгийн санах ойг статик байдлаар байгуулах боломжтой
 - Дэд программыг өөрийн *мэдээллээ* хадгалж байгаа санах ойн хэсэгт статикаар холбодог.
 - *Локал хувьсагч, Процедурын боломжит параметрууд, Буцах хаяг, Түр утгууд*
 - *"Bookkeeping"*: регистрт хадгалагдсан утгууд, дибагт зориулагдсан мэдээлэл ...
 - Нэг процедурын үргэлжилсэн дуудалт нь санах ойгоо дундаа хэрэглэдэг.

СО-н динамик менежмент: Стэк хэрэглээ

- Орчин үеийн програмчлалын ихэнх хэл блок бүтцийг зөвшөөрдөг.
 - Мөрийн эсвэл процедурын блокуудад LIFO схемийг ашиглана
 - А блокт байхад В блокруу орсон бол А-аас гарахын өмнө В-ээс гарсан байна
 - Блок бүрийн локал мэдээллийг хадгалсан стек хэрэглэх тохиромжтой

```
A: { int a = 1;  
      int b = 0;
```

```
      B: { int c = 3;  
            int b = 3;  
          }  
      b = a + 1;  
    }
```



Идэвхжлийн бичлэг

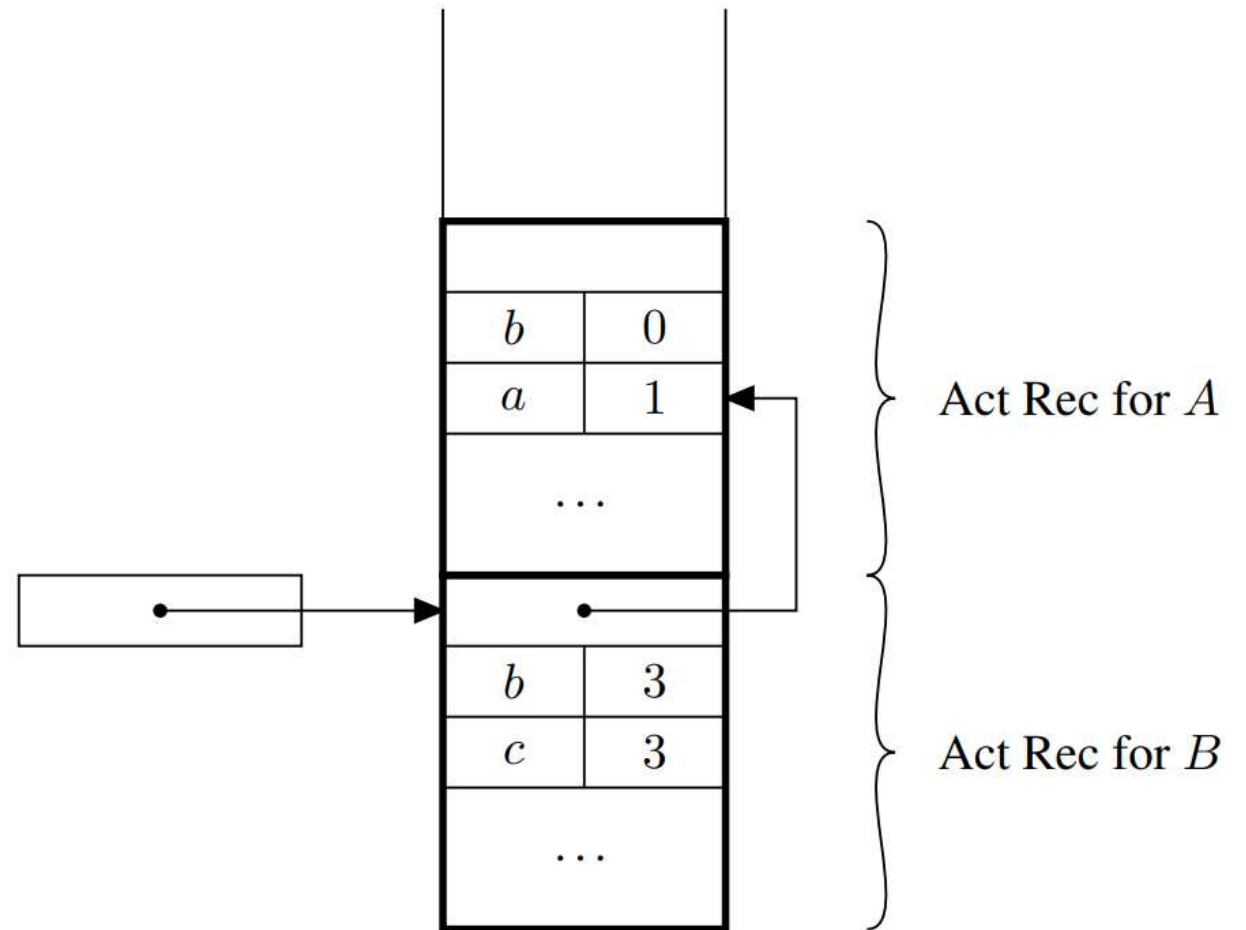
- Ажиллах явцад, А блокт ороход зурагт үзүүлсэн шиг *a* ба *b* хувьсагчдыг багтаах хангалттай том зайг *push* үйлдлээр хуваарилдаг.

```
A: { int a = 1;  
      int b = 0;
```

```
      B: { int c = 3;  
           int b = 3;  
          }  
      b = a + 1;  
    }
```

В блокт орох үед,

- c ба b хувьсагчдад зориулж стек дээр шинэ зайг хуваарилна (дотоод b нь гаднахаас ялгаатай)
- хоёр дахь хуваарилалтын дараах байдлыг зурагт харуулж байна.

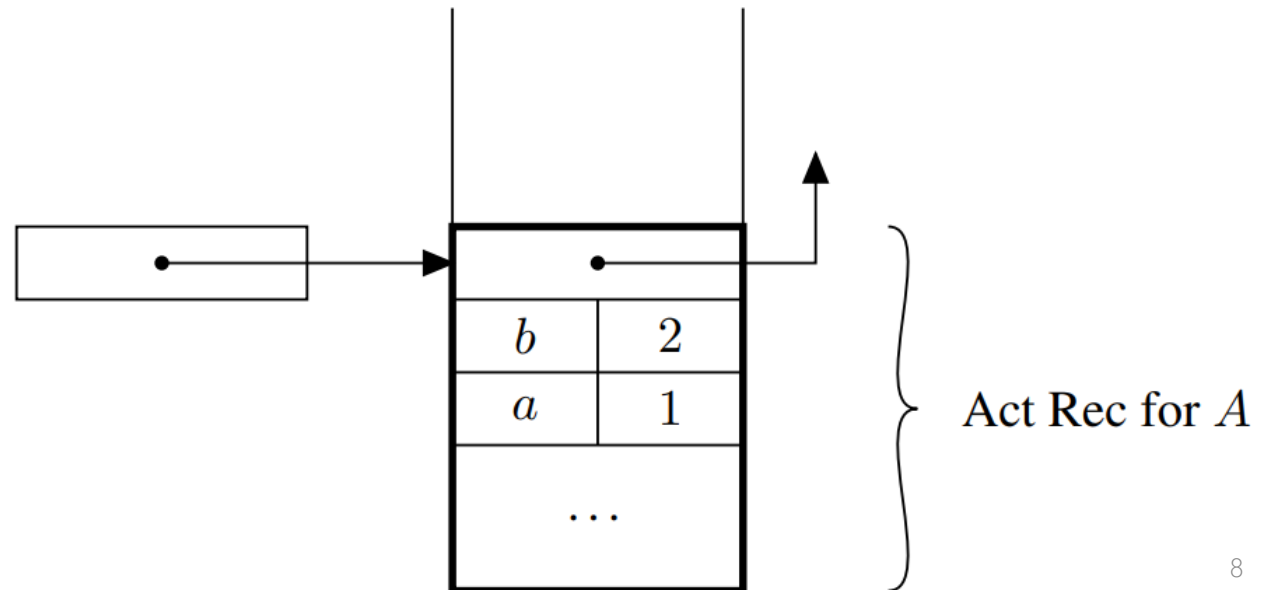


В блокоос гарах үед

- Блокт нөөцөлсөн зайг рор үйлдлээр стекээс гаргаж чөлөөлнө.
- Энэ чөлөөлөлтийн болон утга олголтын дараах байдлыг зурагт үзүүлэв.
- Үүнтэй адилаар, А блокоос гарахад А-ийн зайг багасгахын тулд дахин рор үйлдэл хийнэ.

```
A: { int a = 1;  
      int b = 0;
```

```
      B: { int c = 3;  
           int b = 3;  
          }  
      b = a + 1;  
    }
```

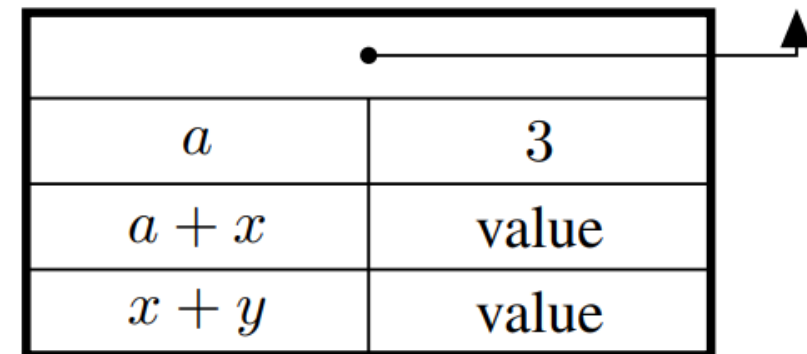


Идэвхжлийн бичлэг: Мөрийн блокт эсвэл процедур идэвхжихэд зориулан стек дээр хуваарилагдсан санах ой. (Фрейм/frame)

- Процедурын зарлалттай холбоотой биш
- Процедурын тодорхой нэг идэвхжилттэй холбоотой
- Биелэлтийн явцад процедурыг дуудах бүрт динамикаар шинийг үүсгэдэг.
- Идэвхжлийн бичлэгт (локал хувьсагч, түр хувьсагч гэх мэт) утгууд нь нэг процедурын өөр өөр дуудалтын хувьд ялгарч болно.
- *Ажиллах үеийн (эсвэл системийн) стек:* Идэвхжлийн бичлэгийг хадгалдаг.
- СО ашиглалтын үүднээс динамик мен-ийг рекурсгүй хэлд ашигладаг.
 - Ижил процедурын идэвхтэй зэрэгцээ дуудалтын дундаж тоо нь програмд зарласан процедурын тооноос бага байвал стек зай хэмнэнэ,
 - учир нь бүхэлдээ статик менежмент тохиолдол шиг зарласан процедур бүрт санах ой хуваарилах шаардлагагүй болно.

- **Завсрын үр дүн** Урьдчилсан тооцоолол хийх хэрэгтэй үед нэргүй ч завсрын үр дүнг хадгална

```
{ int a = 3 ;  
  b = (a + x) / (x + y) ; }
```



a	3
$a + x$	value
$x + y$	value

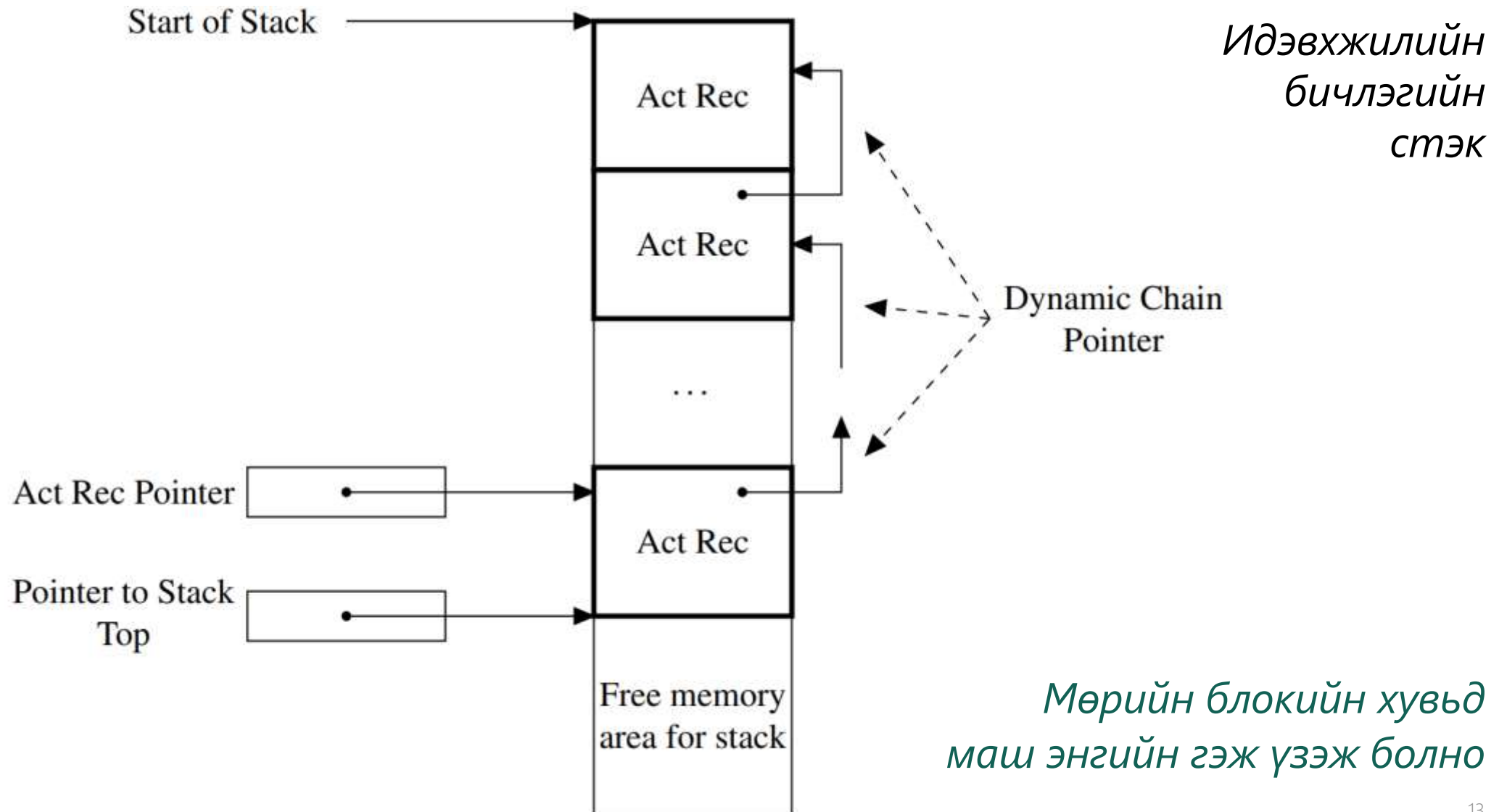
- **Локал хувьсагчид** Блокын дотор зарласан хувьсагчдыг СО-д хадгална
 - Хэмжээг хувьсагчдын тоо, төрлөөс хамааруулан компилятор тооцно.
 - Ажиллах явцын утгаас хамаардаг зарлалт байвал хувьсах урттай хэсгийг агуулна (динамик массив).
- **Динамик гинжин заагч** Стек дээрх өмнөх идэвхжлийн бичлэг (эсвэл хамгийн сүүлд үүссэн идэвхжлийн бичлэг)-ийн заагчийг хадгалдаг.
 - Ерөнхийдөө идэвхжлийн бичлэг өөр өөр хэмжээтэй тул зайлшгүй.
 - Заримдаа *динамик холбоос* эсвэл *хяналтын холбоос* гэж нэрлэдэг.
 - Эдгээр заагчаар бий болсон цувааг *динамик гинж (dynamic chain)* гэнэ.

Функц нь процедураас ялгаатай нь биелэлт дуусах үед дуудагч руу утга буцаадаг (буцаах утгыг хадгалах хаяг).

- **Завсрын үр дүн, локал хувьсагч, динамик гинжийн заагч** Мөрийн блоктой адилхан.
- **Статик гинжин заагч** Энэ нь статик хамрах хүрээний дүрмийг хэрэгжүүлэхэд шаардагдах мэдээллийг хадгалдаг.
- **Буцах хаяг** Тухайн процедур/функцийн биелэлт дууссаны дараа биелэх эхний зааврын хаягийг агуулна.
- **Буцаагдсан үр дүн** Зөвхөн функцэд байдаг. Дэд программ дуусах үед функцийн буцаах утгыг хадгалах санах ойн байршлын хаягийг агуулна. Энэ байршил нь дуудагчийн идэвхжлийн бичлэг дотор байна.
- **Параметрууд** Процедур эсвэл функцийг дуудахад ашигласан бодит параметруудийн утгыг энд хадгална.

- Идэвхжлийн бичлэгийн талбарууд нь хэрэгжилтээс бүрд ялгатай.
 - Заагч нь идэвхжлийн бичлэгийн тогтмол (ихэвчлэн төв) мужийг заадаг.
 - Талбарын хаяг = заагчийн утга + эерэг/сөрөг оффсет
- Идэвхжлийн бичлэгт хувьсагчийн нэрийг шууд хадгалдаггүй
 - Локал хувьсагчийг зарласан блокын идэвхжлийн бичлэгийн тогтмол байрлалд харьцангуй хаягаар (оффсет) компилятор орлуулдаг.
- Нон-локал хувьсагчийн хувьд нэр хадгалахаас зайлсхийдэг механизмудыг ашиглах боломжтой
 - Идэвхжлийн бичлэгийн стэк дээр ажиллах үед нэрийг хайхаас сэргийлэх.
- Орчин үеийн компиляторууд идэвхжлийн бичлэгийн оронд зарим мэдээллийг регистрт хадгалан үүсгэсэн кодондоо оновчлол хийдэг.
 - Ойлгомжтой байх үүднээс эдгээр оновчлолыг бид авч үзэхгүй.

ДИНАМИК МЕНЕЖМЕНТ, СТЭК ➤ Стэк менежмент



Дуудагч (*Caller*) болон Дуудагдагч (*Callee*) хоёул стек менежментийг гүйцэтгэгддэг.

- Дуудагч талд *дуудалтын дараалал (calling sequence)* код нэмэгддэг
 - Процедур дуудалтын яг өмнө дуудалтын дарааллын нэг хэсэг код биелэгддэг
 - Үлдсэн хэсэг нь дуудалт дууссаны дараа шууд биелэгдэнэ.
- Дуудагдагчид хоёр код нэмэгддэг:
 - Дуудалт руу орсны дараа шууд биелэгдэх *пролог (prologue)*,
 - Процедур дуусахаас өмнө хийгдэх *эпилог (epilogue)*.
- Эдгээр гурван хэсэг нь идэвхжлийн бичлэгтэй ажиллах, процедурын дуудалтыг зөв хэрэгжүүлэхэд шаардлагатай үйлдлүүдийг удирддаг.

- **Программын тоолуур өөрчлөлт** Дуудагдсан процедурт хяналтыг шилжүүлнэ. Буцах хаягт хуучин утгыг (+1) хадгална.
- **Стек зайн хуваарилалт** Идэвхжлийн шинэ бичлэгийн зайг урьдчилан хуваарилах; сул зайн заагчийг шинэчилэх.
- **Идэвхжлийн бичлэг заагчийг засах** дуудагдсан процедурын шинэ идэвхжлийн бичлэгийг заана: идэвхжлийн бичлэгийг стек рүү оруулна.
- **Параметр дамжуулалт** Нэг процедурын өөр өөр дуудалт өөр өөр параметртэй байж болох тул энэ үйлдлийг ихэвчлэн дуудагч гүйцэтгэдэг.
- **Регистрт хадгалах** Хяналтын боловсруулалтад зориулсан утгуудыг хадгалах ёстой, ерөнхийдөө регистрт хадгалдаг. Жнь: динамик гинжийн заагчаар хадгалсан идэвхжлийн хуучин бичлэгийн заагч.
- **Эхлэх кодыг биелэлт** Шинэ идэвхжлийн бичлэгт хадгалагдсан зүйлсийг эхлүүлэхийн тулд байгуулахыг зарим хэл шаарддаг.

- **Программын тоолуур өөрчлөлт** Дуудагчид хяналтыг буцан шилжүүлнэ.
- **Утга буцаалт** Дуудлагчаас процедурт мэдээлэл дамжуулсан параметрийн утгууд дуудагчийн идэвхжлийн бичлэгт хадгалагдах ёстой.
- **Регистрыг буцаах** Өмнө нь хадгалсан регистрийн утгыг сэргээх ёстой.
 - Ялангуяа идэвхжлийн бичлэг заагчийн хуучин утгыг сэргээх ёстой.
- **Дуусгах кодын биелэлт** Зарим хэл нь бүх локал объект устгагдахаас өмнө тохиромжтой дуусгах кодыг биелүүлэхийг шаарддаг.
- **Стек зайн чөлөөлөлт** Дууссан процедурын идэвхжлийн бичлэгийг стекээс устгах ёстой.
 - Стекийн заагчийг (сул зайн эхний байрлал) өөрчлөх шаардлагатай.

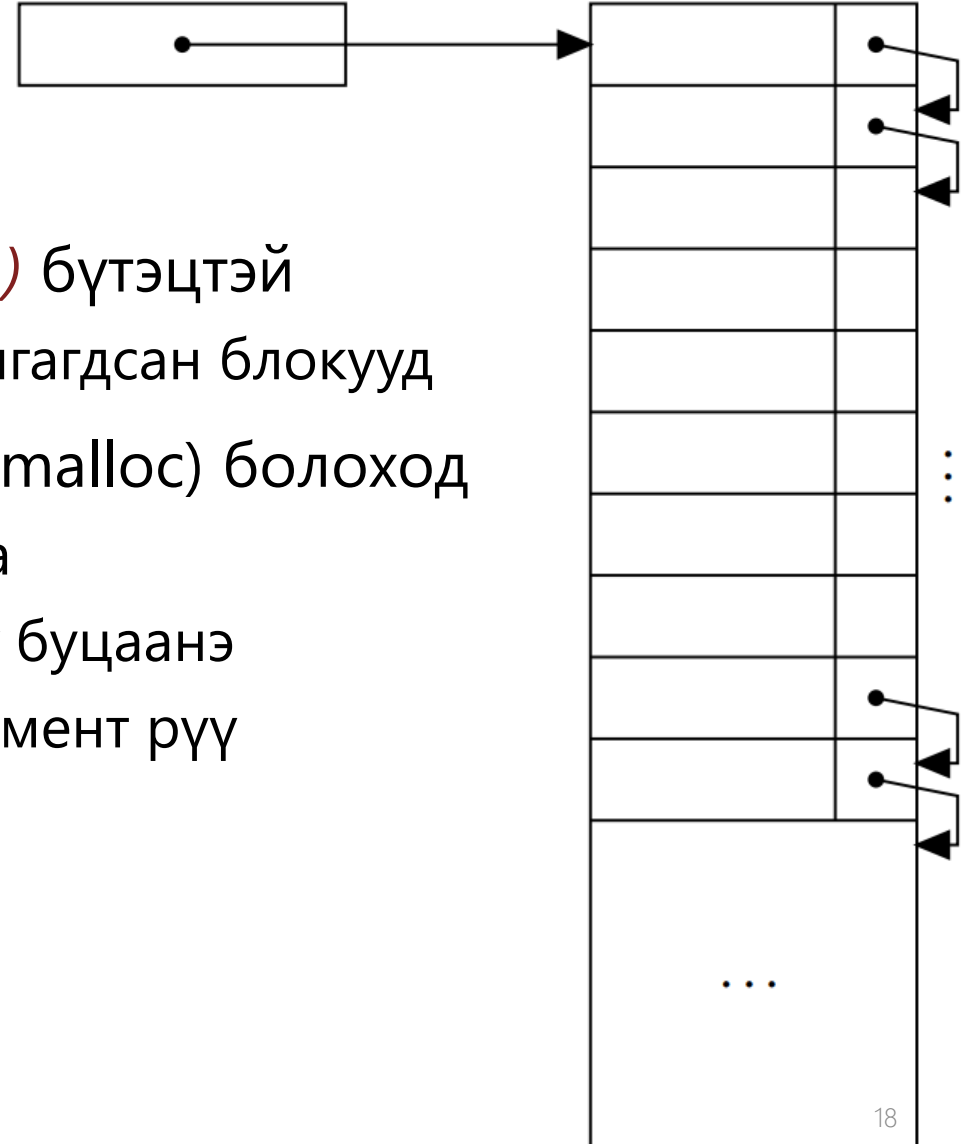
СО-н динамик менежмент: Овоолго хэрэглээ

- СО хуваарилалтын ил командтай хэлний хувьд зөвхөн стек хангалтгүй.
 - p, q –ийг ижил дарааллаар хуваарилж, чөлөөлөхгүй (жнь: C, Паскал...)

```
int *p, *q;           /* p,q NULL pointers to integers */
p = malloc (sizeof ( int )); /* allocates the memory pointed to by p */
q = malloc (sizeof ( int )); /* allocates the memory pointed to by q */
*p = 0;              /* dereferences and assigns */
*q = 1;              /* dereferences and assigns */
free(p);             /* deallocates the memory pointed to by p */
free(q);             /* deallocates the memory pointed to by q */
```

- Ямар үед гарч болох ил хуваарилтанд **овоолго (heap)** ашигладаг
 - Өгөгдлийн бүтцийн (эрэмбийн үр ашиг) овоолготой ямар ч “хамаагүй шд”
 - Харьцангуй чөлөөтэй хуваарилах, чөлөөлөх зорилготой
 - СО-ны блокуудыг **тогтмол**, эсвэл **хувьсах** хэмжээтэй байхаар авч үздэг

FL Start



- Овоолго нь *чөлөөт жагсаалт (Free list – FL)* бүтэцтэй
 - тодорхой тооны ижил жижиг хэмжээтэй залгагдсан блокууд
- Санах ойн блок хуваарилах шаардлагатай (malloc) болоход
 - Чөлөөт жагсаалтын эхний элементийг хасна
 - Заагчийг нь санах ойг шаардсан үйлдэл рүү буцаанэ
 - Чөлөөт жагсаалтын заагчийг дараагийн элемент рүү шилжүүлж шинэчилнэ.

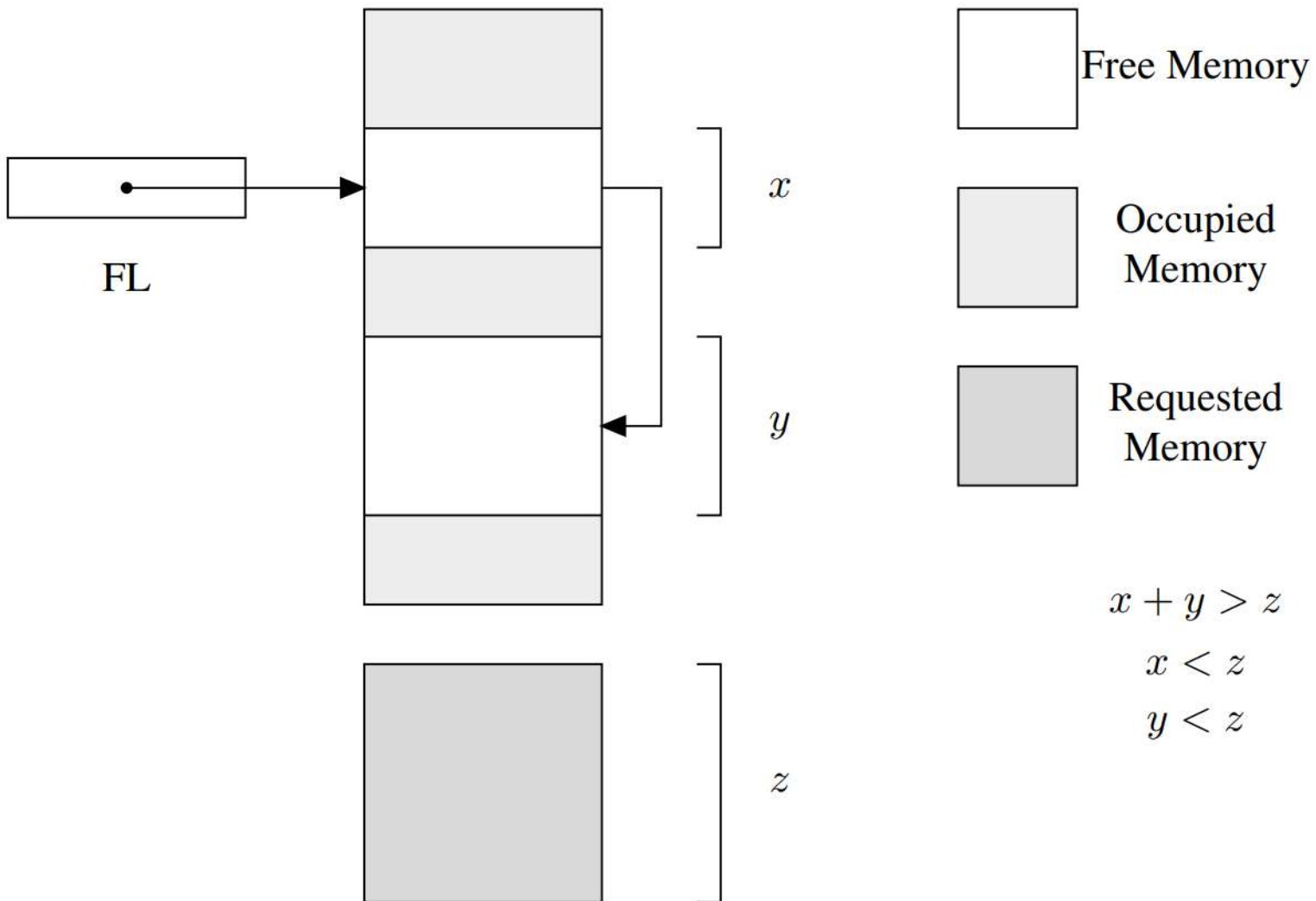
FL Start



- Санах ойг суллах (freed) эсвэл чөлөөлөх (deallocated) үед суларсан блок нь чөлөөт жагсаалтын толгойд дахин холбогддог.
- Дараах тохиолдолд маш тогтмол хэмжээтэй блокуудыг удирдах нь хялбар
 - Ажиллах үед (runtime), буцаах ёстой санах ойг *ялган таних* болон *дахин холбохыг* мэддэг байх
- Гэхдээ эдгээр үйлдлүүд нь дандаа тодорхой байхгүй

- Хувьсах урттай массив байхад СО нь тогтмол блокийн хэмжээнээс их байхыг шаардах бөгөөд санах ойн үргэлжилсэн бүс шаардлагатай
 - Блок цуваагаар хуваарилах боломжгүй (үргэлжилсэн байж чадахгүй).
 - Хувьсах хэмжээтэй блокой овоолгод суурилсан схемийг ашигладаг.
- Овоолго менежмент үйлдүүд нь ерөнхийдөө программын **СО ашиглалт** болон **биелэлтийн хурдыг** нэмэгдүүлэх зорилготой
 - Ажиллах үед гүйцэтгэдэг тул програм биелэлтийн хугацаанд нөлөөлдөг
 - Ихэнхдээ, эдгээр хоёр шинж чанарыг нийцүүлэх хэцүү
- СО ашиглалтын хувьд **хэсэглэл (fragmentation)** үүсэхээс зайлсхийх
 - *Дотоод (Internal)*: Блок нь шаардсанаас том. Чөлөөлөгдөх хүртэл зай үүснэ.
 - *Гадаад (External)*: Сул блокуудын нийт хэмжээ шаардсанаас том байх боловч үргэлжилсэн байж чадахгүй байгаа бол илүү ноцтой

ДИНАМИК МЕНЕЖМЕНТ, ОВООЛГО» Гадаад хэсэгчлэл



- СО хуваарилалтын техникүүд нь чөлөөт санах ойг "нягт (compact)" байлгахыг зорьдог
- Зэргэлдээ сул блокуудыг нэгтгэдэг.
- Менежментийн аргуудын ачаалалтыг нэмэгдүүлж, үр ашгийг бууруулдаг.

- Эхлээд овоолгыг бүтнээр агуулсан санах ойн нэг блок гэж үздэг.
 - Боломжит том блокыг олох (Эхлээд олон жижиг блокт хуваах нь утгагүй)
- Санах ойн n үгтэй блок хуваарилах шаардлагатай үед
 - Эхний n үгийг хуваарилж, эхлэлийн заагчийг n -ээр нэмэгдүүлнэ.
 - Дараа, дараагийн хүсэлтийг ижил байдлаар гүйцэтгэдэг.
 - Чөлөөлөгдсөн блокуудыг чөлөөт жагсаалтад цуглуулдаг.
- Овоолгын СО-н төгсгөлд ирэхэд чөлөөлөгдсөн СО-г ашиглана:
 - Чөлөөт жагсаалтыг шууд ашиглах** Хувьсах хэмжээтэй блокуудын чөлөөт жагсаалтыг ашиглана. Тохирох (*first fit, best fit*) блокын илүү хэсгийг салгаж чөлөөт жагсаалтанд нэгтгэнэ (using threshold). Чөлөөлөгдөх үед гадаад хэсэгчлэлээс сэргийлж зэргэлдээ блок нь сул бол нэгтгэнэ.
 - Чөлөөт санах ойг нягтруулах** Бүх идэвхтэй хэвээр блокуудыг овоолгыг төгсгөлд шилжүүлнэ. Буцаад чөлөөт блок болохгүй, нэг бүхэл санах ойн нэг үргэлжилсэн блокт нэгдэнэ.

- Блок хуваарилалтын зардлыг бууруулахын тулд өөр өөр хэмжээтэй блок агуудсан чөлөөт жагсаалтуудыг ашигладаг.
 - n хэмжээтэй блок хүсэх үед n -ээс их буюу тэнцүү хэмжээтэй блок агуулсан жагсаалтын блокыг сонгоно (жижиг дотоод хэсэглэлтэй).
 - Блокын хэмжээ нь статик эсвэл динамик байж болно.
- Динамик хэмжээтэй тохиолдолд удирдлагын хоёр арга ашигладаг:
 - **Найзын систем:** Чөлөөт жагсаалтууд 2^k блок хэмжээтэй. Багтах чөлөөт блок 2^k жагсаалтанд байхгүй бол 2^{k+1} жагсаалт блокыг 2 хуваан хуваарилж, зөрүү 2^k -г чөлөөт жагсаалтад оруулна. Чөлөөлөгдөх үед түүний "найз"-ыг нь олоод буцаан нэгтгэж 2^{k+1} хэмжээтэй анхны блокыг үүсгэнэ.
 - **Фибоначч овоолго:** ижил төстэй боловч 2^k -ийн Фибоначчийн тоог ашигладаг. Фибоначчийн дараалал нь 2^n цуваанаас илүү удаан өсдөг тул энэ хоёр дахь арга нь дотоод хэсэглэл багатай байдаг.



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА